DSA STUDY BLUEPRINT SERIES

Product of Array Except Self

Division-free Prefix and Suffix Sweeps

Section 08 • C++ Core Data Structures & Algorithms

The Constraint Breakdown

Evaluating limitations: O(N) time and no division operator

Why Not Use Division?

- **Division Method:** Multiply all elements together, then divide total by `nums[i]`.
- **Fatal Flaw:** Fails if array contains `0` (Division by zero crash).
- Requires separate checks for single or multiple zeroes.

The Split Sweep Intuition

- Any element's product except itself is: `Prefix Product * Suffix Product`.
- `Prefix Product[i]` = product of all elements to the left of *i*.
- `Suffix Product[i]` = product of all elements to the right of *i*.

Algorithm Execution & Optimization

Reducing space complexity from O(N) auxiliary to O(1)

Two Pass Sweep Logic

We can reuse the **output array** to store prefix products, avoiding extra allocations.

1. **Prefix Sweep:** Calculate prefix products and store directly in `ans[i]`.
2. **Suffix Sweep:** Maintain a variable `suffix = 1`. Multiply `ans[i]` by `suffix`, then update `suffix *= nums[i]` moving backwards.

State Trace: [1, 2, 3, 4]

1. Prefix Sweep:

- `ans = [1, 1, 2, 6]`

2. Suffix Sweep (backwards, suffix starts at 1):

- i = 3: `ans[3] *= 1` → 6 | `suffix = 1 * 4 = 4`
- i = 2: `ans[2] *= 4` → 8 | `suffix = 4 * 3 = 12`
- i = 1: `ans[1] *= 12` → 12 | `suffix = 12 * 2 = 24`
- i = 0: `ans[0] *= 24` → 24

Result: `[24, 12, 8, 6]`

Prefix/Suffix Sweep C++ Code

O(N) Time and O(1) Auxiliary Space implementation

```
#include
#include
using namespace std;

vector productExceptSelf(vector& nums) {
    int n = nums.size();
    vector ans(n, 1);

    // Step 1: Prefix products
    for (int i = 1; i < n; i++) {
        ans[i] = ans[i - 1] * nums[i - 1];
    }

    // Step 2: Suffix products (in-place)
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        ans[i] *= suffix;
        suffix *= nums[i];
    }

    return ans;
}
```